GUÍA DE PROYECTO:

DISEÑO, CONSTRUCCIÓN Y CONTROL DE UN PROTOTIPO AUTÓNOMO DE EQUILIBRIO DINÁMICO PARA SEGUIMIENTO DE TRAYECTORIA (“ROBOT SEGUIDOR BALANCÍN”)

Fase de Diseño, Construcción y Control de Equilibrio
(Corte 1)

Grupo:
Ivan Kaleth Briceño Rojas 1093589495
Jaiver Andrés Tolosa velandia 1090539815
Gibran Schneider Guerrero Rolon 1091353836
Yohan Manuel Oviedo Padilla 1126707080

Docente:
Ing. Jeisson Martínez Flórez

Asignatura:
Control Inteligente
II

Universidad de Pamplona
Facultad de Ingenierías y Arquitectura
Programa de Ingeniería Mecatrónica
Villa del Rosario

16 de Octubre 2026

TABLA DE CONTENIDO

Contenido

GUÍA DE PROYECTO: 1

Fase de Diseño, Construcción y Control de Equilibrio (Corte 1) 1

Docente: 1

TABLA DE CONTENIDO 2

1. ESPECIFICACIÓN GENERAL DEL PROYECTO 3

1.1. Resumen 3

1.2. Objetivos Específicos. 3

1.3. Alcance y Limitaciones 3

2. ANÁLISIS DEL SISTEMA DE CONTROL DE EQUILIBRIO 4

3. DISEÑO 5

3.1. Diseño Mecánico CAD 3D 5

3.2. Selección de Materiales 7

3.3. Esquema de Circuito Electrónico 11

3.4. Estimación de costos 12

4. CONSTRUCCIÓN 12

Ensamblaje Estructural y Montaje del Robot Balancín 12

Materiales Estructurales Base 12

Estructura por Niveles 12

5. IMPLEMENTACIÓN DEL CONTROL DE EQUILIBRIO 16

5.1. Diagrama de Flujo del Algoritmo Implementado 16

5.2. Desglose del Código del Algoritmo 18

5.2.1. Librerías (#include) 18

5.2.2. Configuración Wi-Fi y WebSockets 18

5.2.3. Asignación de pines y parámetros físicos del carro 18

5.2.4. Registros y estructura de datos del MPU-6050 19

5.2.5. Parámetros de control, offset físico y variables globales 20

5.2.6. Funciones de bajo nivel: encoders, I2C y lectura del sensor 21

5.2.7. Control de motores 22

5.2.8. Recepción de comandos desde la web 23

5.2.9. Tarea de control principal (TaskBalanceControl) 24

5.2.10. Función setup() 26

5.2.11. Función loop(): WebSocket y telemetría 27

5.3. Resumen de las Etapas del Lazo de Control 28

REFERENCIAS. 29

1. ESPECIFICACIÓN GENERAL DEL PROYECTO

1.1. Resumen

El proyecto consiste en el diseño, construcción y control de un **robot balancín autónomo** capaz de mantener su equilibrio dinámico mientras sigue una trayectoria. El prototipo está compuesto por una estructura impresa en 3D, dos motorreductores con encoders, un sensor MPU-6050 que mide la inclinación en tiempo real, y un controlador **ESP32 S3**, el cual procesa los datos y aplica una **estrategia de control PID** para mantener la estabilidad ajustando la potencia de los motores mediante señales PWM. El **objetivo final** es lograr que el robot se equilibre por sí mismo y pueda desplazarse de forma estable, evidenciando la correcta implementación de un control inteligente. Entre los **principales desafíos** enfrentados se encuentran la selección adecuada de los motores y materiales dentro de las limitaciones de tamaño y peso, la calibración del sensor MPU-6050, y la sintonización manual de las constantes del PID, proceso que requirió múltiples pruebas hasta obtener una respuesta estable y precisa.

1.2. Objetivos Específicos.

- Estudiar la diversidad de componentes que podría usar el robot.
- Seleccionar los mejores componentes de acuerdo a su disponibilidad.
- Diseñar el cuerpo del robot y circuito eléctrico cumpliendo las medidas establecidas.
- Realizar el control PID por medio de prueba y error.

1.3. Alcance y Limitaciones

De acuerdo al reglamento las dimensiones de dicho robot no deben superar los 20 cm x 20 cm x 20 cm por lo que a la hora de selección de componentes y diseño del robot deben adaptarse a dicho reglamento. Además de este su peso no debe superar a 1 kilogramos, por lo que se debe tener en cuenta los pesos de los motores, llantas, batería y cuerpo, ya que son los que contienen el peso más significativo por lo que se ha mantenido en estos rangos.

2. ANÁLISIS DEL SISTEMA DE CONTROL DE EQUILIBRIO

Pregunta	Respuesta Detallada y Justificada
Objetivo de Control	Evitar que el robot pierda el equilibrio y caiga al suelo.
Variable Controlada	El robot controlara el Angulo de inclinación de esta misma y se estará midiendo en grados de inclinación de tal manera que el usuario pueda identificar mejor la varianza de esta.
Variable Manipulada	Para lograr el control del robot se manipulará la fuerza que ejercen cada uno de los motorreductores de acuerdo a la dirección de inclinación del robot enviándole señales en PWM.
Sensor	Se hará uso de un módulo MPU-6050 por su bajo costo y alta integración por su que combina en un solo chip un acelerómetro triaxial y un giroscopio triaxial.
Actuador	El efector final son unos Motorreductores de 12V y 300 rpm, teniendo así un buen torque para su control.
Controlador	En controlador que se estará usando es un módulo esp32 s3, por su alto rendimiento y versatilidad, ya que integra un procesador dual-core de 240 MHz, Wi-Fi y Bluetooth, y 16 MB de flash con 8 MB de PSRAM
Alimentación / Energía	Se utilizarán 3 baterías de litio 18650 en serie conectadas a un módulo BMS s3 que cuidara de las baterías las cuales serán para alimentar directamente los motores y con un regulador de voltaje pueda bajar el voltaje para alimentar el resto de la placa.
Interfaz Humano Máquina	El robot será programado en lenguaje C con una interfaz web está construida con la combinación estándar del desarrollo web: HTML, CSS y JavaScript, integrando además herramientas específicas para gráficos
Setpoint	El setpoint será asignado directamente de la programación con la interfaz de Arduino.
Error	Con el módulo MPU detectará su ángulo en tiempo real y está comparándola con el ángulo deseado ingresado en la programación.
Rango de Operación	Se desea que el robot opere en un rango de 35 a -35 grados logrando un control efectivo.
Estrategia de Control	La estrategia de control que se implementara es control en cascada (a dos frecuencias de muestreo) complementado con un lazo desacoplado de orientación

3. DISEÑO

3.1. Diseño Mecánico CAD 3D

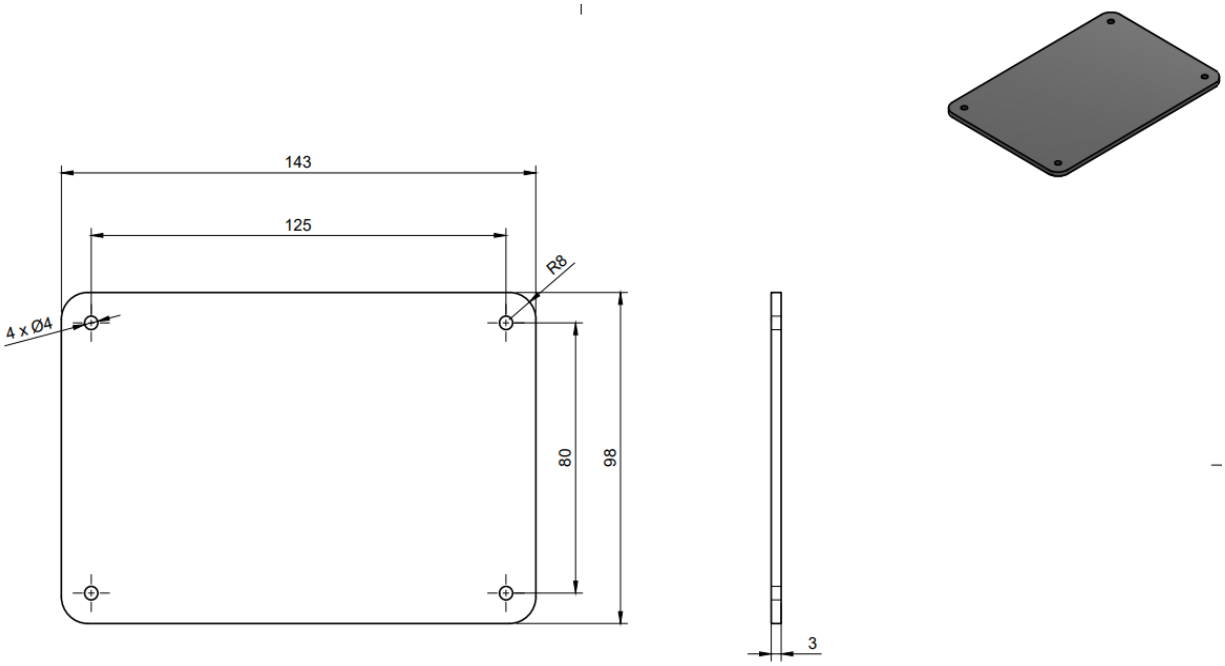


Ilustración 1. Tapa del robot.

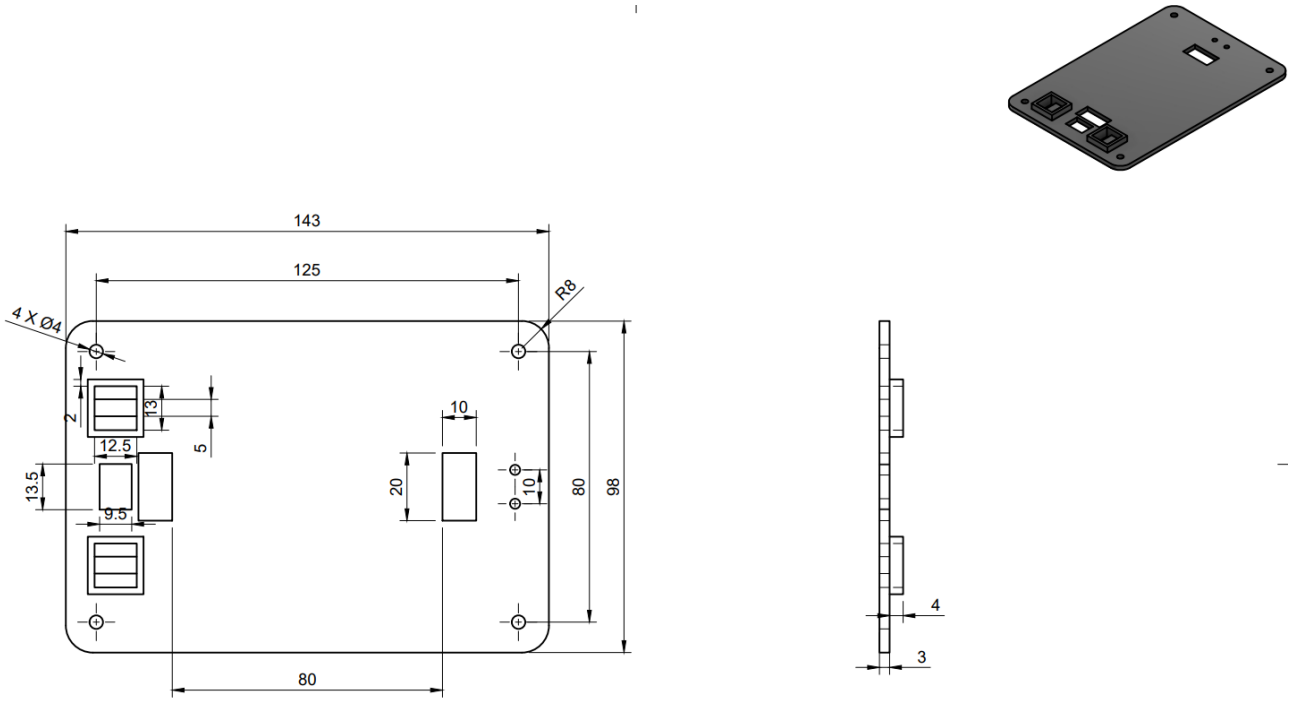


Ilustración 2. Tapa de la pcb.

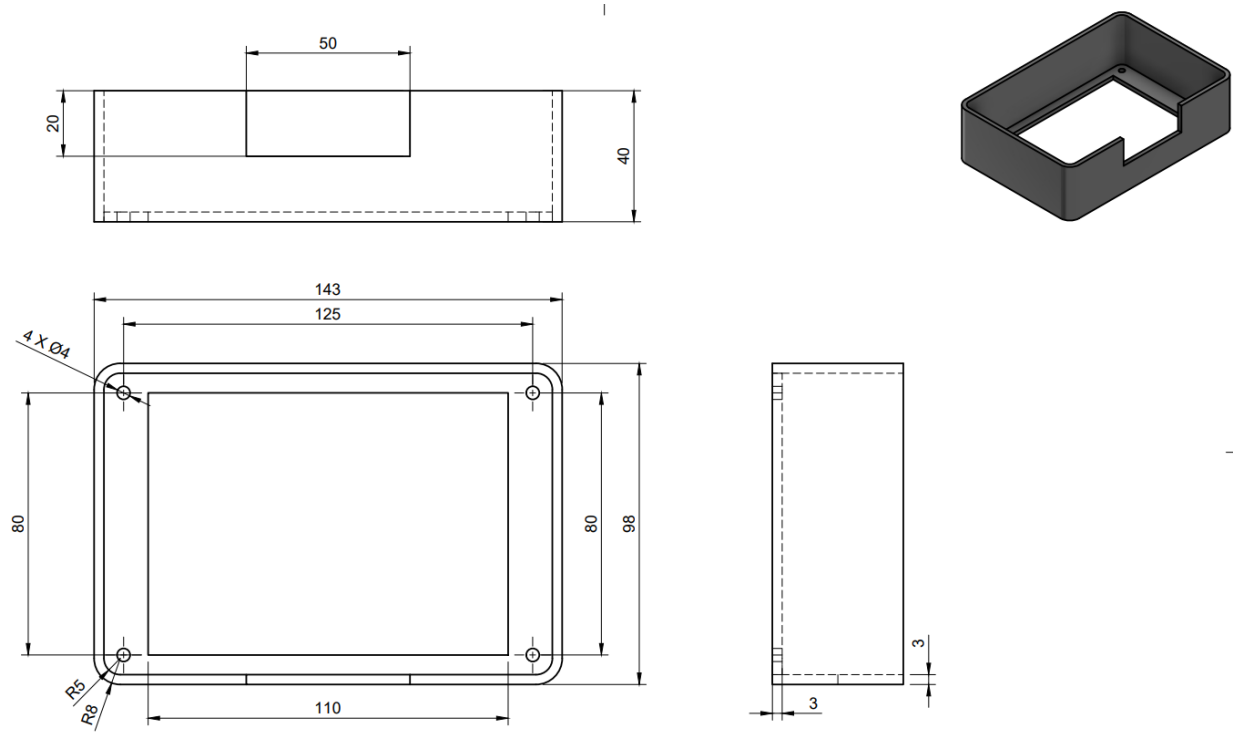


Ilustración 4. Base pcb.

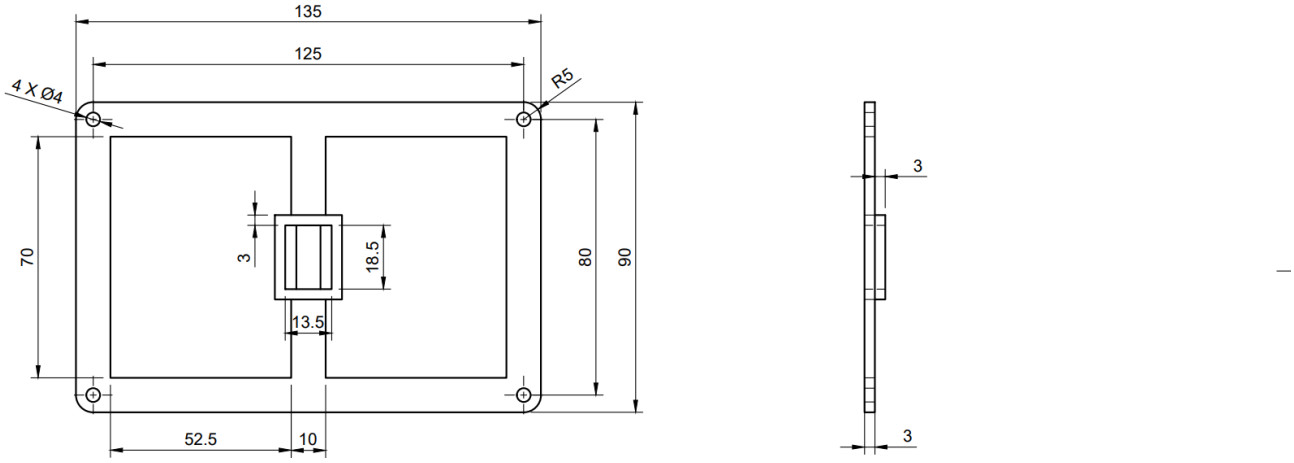
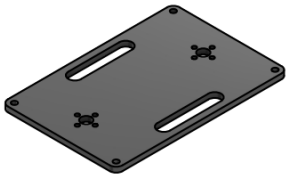


Ilustración 3. Soporte del giroscopio.



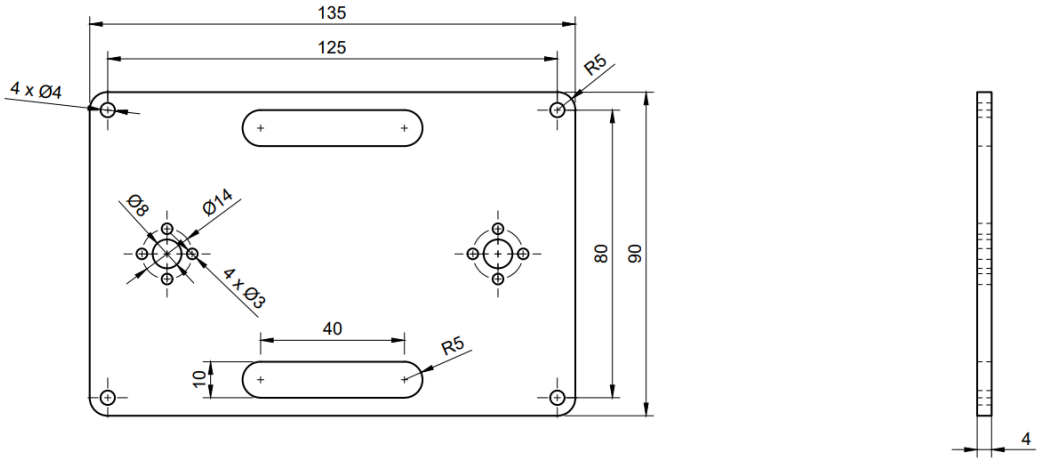


Ilustración 5. Base principal.

3.2. Selección de Materiales

- **Esp32 s3:** Este controlador fue seleccionado por su alto rendimiento y versatilidad, ya que integra un procesador dual-core de 240 MHz, Wi-Fi y Bluetooth, y 16 MB de flash con 8 MB de PSRAM, siendo esta más rápida que el Arduino Uno.



Motorreductor: 25GA370-12V-300RPM Con encoder. un motorreductor con encoder para generar un mayor control en el robot conociendo los pulsos generados por esta. Además, presentan un alto torque para un control más preciso.



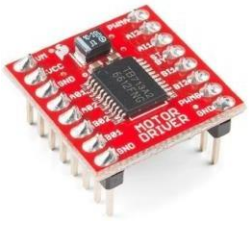
- **Llanta de lujo:** Fue seleccionado por su precisión de acople con el motor seleccionado.



- **Modulo MPU-6050**: Este módulo se seleccionó por su bajo costo y alta integración que combina en un solo chip un acelerómetro triaxial y un giroscopio triaxial.



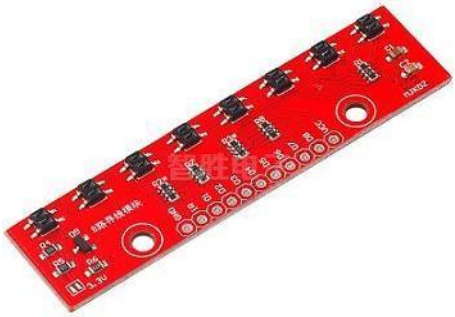
- **TB6612FNG**: Este driver tiene un pequeño tamaño y bajo costo por lo que es ideal para este diseño. Fue utilizados dos drivers para evitar sobrecalentamiento en momentos críticos.



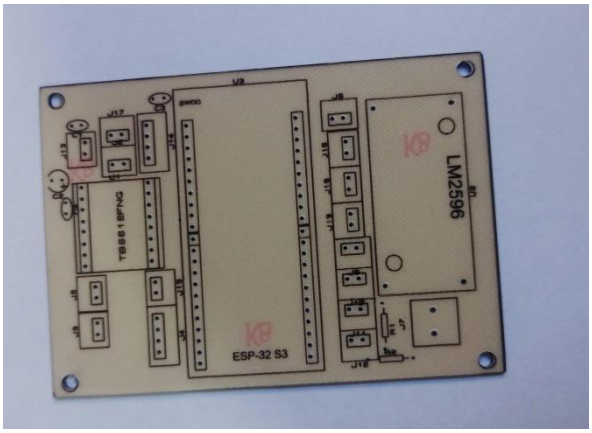
- **LM2596**: Su selección es por su bajo costo y buena accesibilidad.



- **Sensor seguidor de Línea 8 canales**: Se selecciono por su disponibilidad.



- **PCB:** Se recurrió realizar una Pcb ya que contiene gran cantidad de conexiones y por el tamaño comparto y organizado que el robot requiere.



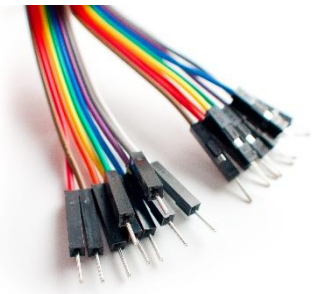
- **PLA:** Se utilizo este material ya que es ligero, cómodo y de buena durabilidad para este robot.



Otros accesorios:

- Bornes, Jumpers, conectores hembra, interruptor on/off, boton de inicio, conectores kf2510, terminales, tornillos 3mm para su ensamble.











3.3. Esquema de Circuito Electrónico

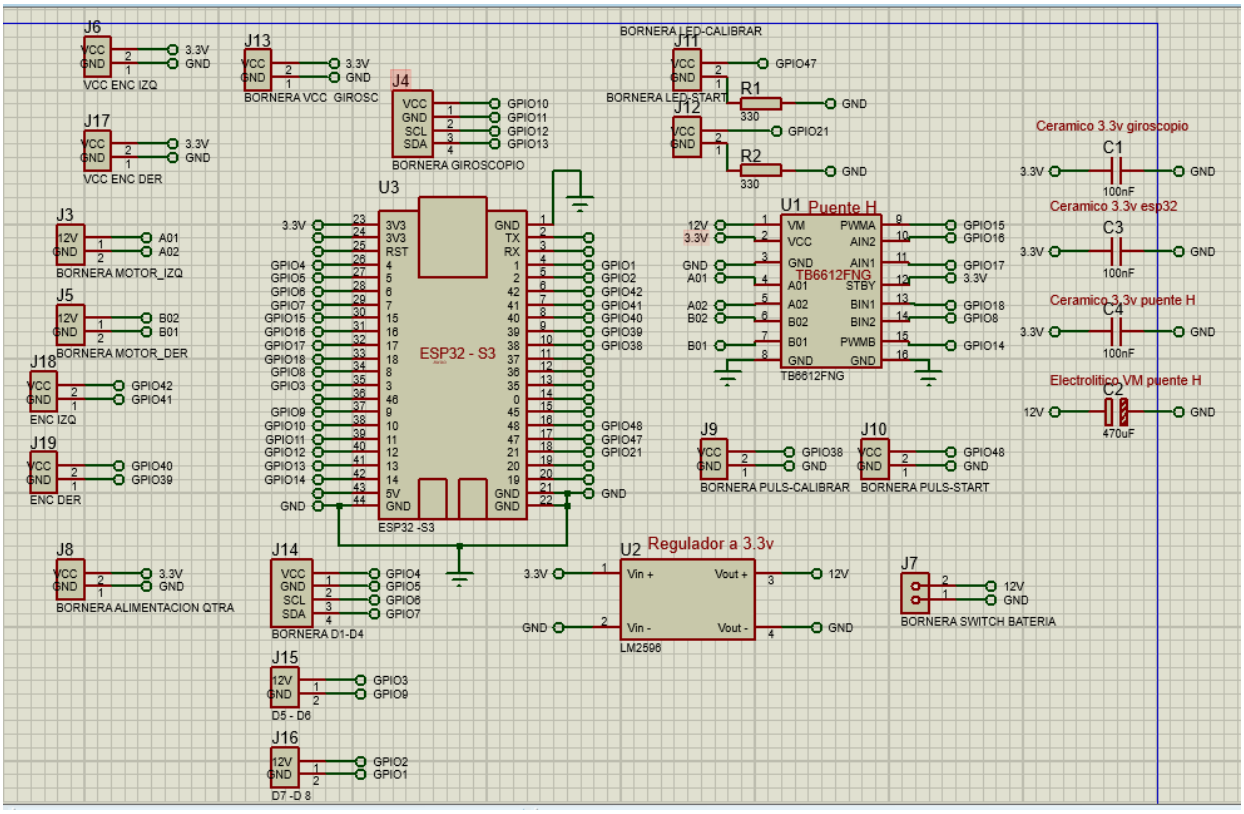


Ilustración 6. Esquema de la pcb.

La ilustración 5 de presenta las conexiones con lo que se realizara la pcb ya que con esta ira conectada con todos los componentes del robot haciéndola más compacta.

3.4. Estimación de costos

Productos	Cantidad	Precio unitario	Impuestos	Importe
[TRACKING-8-CH] Sensor seguidor de línea- 8 canales	1,00 Unidades	$ 7.247,00	19% IVA	$ 7.247,00
[25GA370-12V-300RPM] Motorreductor 12V 300RPM. Con encoder	2,00 Unidades	$ 31.354,00	19% IVA	$ 62.708,00
[F10003-AZ] Rueda Plástica con rin azul	2,00 Unidades	$ 2.816,00	19% IVA	$ 5.632,00
[LM2596-V-5A] Conversor DC/DC Reductor 0 -24v 5A	1,00 Unidades	$ 4.752,00	19% IVA	$ 4.752,00
[TB6612FNG] Driver Dual Motor DC TB6612FNG	1,00 Unidades	$ 6.336,00	19% IVA	$ 6.336,00
[AR-DI4L25-4x6] Acople Rígido para Motor de 4mm a 6mm	2,00 Unidades	$ 3.993,00	19% IVA	$ 7.986,00
[GY-BMI160] Acelerómetro, giroscopio BMI160	1,00 Unidades	$ 9.152,00	19% IVA	$ 9.152,00
Impresiones	1,00 Unidades	$ 62.000,00	19% IVA	$ 62.000,00
acrilico	1,00 Unidades	$ 10.000,00	19% IVA	$ 10.000,00
batería litio 18650	1,00 Unidades	$ 20.000,00	19% IVA	$ 20.000,00
soporte y modulo BMS S3	1,00 Unidades	$ 25.000,00	19% IVA	$ 25.000,00
tornillería	1,00 Unidades	$ 15.000,00	19% IVA	$ 15.000,00
placa PCB	1,00 Unidades	$ 35.000,00	19% IVA	$ 35.000,00
Transportadora	1,00 Unidades	$ 16.600,00	19% IVA	$ 16.600,00

Importe base	$ 287.413,00
IVA 19%	$ 54.608,47
Total	**$ 342.021,47**

4. CONSTRUCCIÓN

Ensamblaje Estructural y Montaje del Robot Balancín

El diseño mecánico del robot balancín se planteó con una arquitectura vertical de múltiples niveles, utilizando varillas roscadas para garantizar rigidez, modularidad y facilidad de ajuste en la altura de los componentes.

A continuación, se detalla el proceso de montaje estructural dividido en cuatro niveles principales, construidos sobre un chasis soportado por varillas de rosca sin fin de 3/16 de pulgada.

Materiales Estructurales Base

Varillas de rosca sin fin (3/16 pulgada).
Tuercas y arandelas de 3/16 para fijación y soporte de niveles.
Bases/Plataformas (incluyendo una base superior de acrílico).

Estructura por Niveles

Nivel 1: Chasis Base y Sistema de Tracción
El primer nivel conforma la base principal del robot. En este piso inferior se encuentran anclados los actuadores del sistema.

Montaje: Se fijaron los dos motores de tracción directamente a la base inferior. Cada motor está asegurado firmemente con dos tornillos para evitar vibraciones u holguras que puedan afectar el control de equilibrio.
Función: Proporcionar el movimiento de las ruedas y soportar el peso de toda la estructura superior.
Nivel 2: Sistema Inercial y Centro de Gravedad
Para elevar la estructura, se introdujeron las varillas roscadas de 3/16 en las esquinas de la base. Utilizando dos tuercas del diámetro de las varillas por cada punto de apoyo, se "aprisionó" el piso anterior y se le dio el soporte nivelado a este nuevo piso.

Componente principal: En este nivel se ubicó estratégicamente el módulo acelerómetro y giroscopio MPU6050.
Función: Al estar en un nivel intermedio e inferior, permite leer con mayor precisión la inclinación y orientación del carrito balancín, datos vitales para el algoritmo de control.
Nivel 3: Etapa de Control y Potencia
Continuando con la fijación mediante tuercas a través de las varillas roscadas, se estableció el tercer nivel, considerado el "cerebro" y "corazón" del robot.

Caja de Control: Aquí se aloja la placa de circuito impreso (PCB) principal que contiene el controlador (ESP) y todo el circuito de potencia y control de los motores.
Alimentación: Justo encima de la etapa de control, en este mismo nivel, se dispuso el sistema de energía documentado previamente: el portapilas con las 3 baterías de litio 18650 (Configuración 3S) junto con su módulo de protección BMS.

Nivel 4: Plataforma de Carga (Techo Acrílico)
El último nivel cierra la estructura del chasis en la parte superior.

Montaje: Se utilizó una base de acrílico transparente para brindar un mejor acabado estético y mayor limpieza visual.

Función Práctica: Este "techo" no solo protege la electrónica inferior, sino que fue diseñado intencionalmente como una plataforma plana. Sirve de base para colocar objetos sobre el robot, permitiendo realizar pruebas dinámicas ("juegos") donde el robot debe mantener el equilibrio mientras soporta carga extra en su parte más alta, desafiando el algoritmo de control.

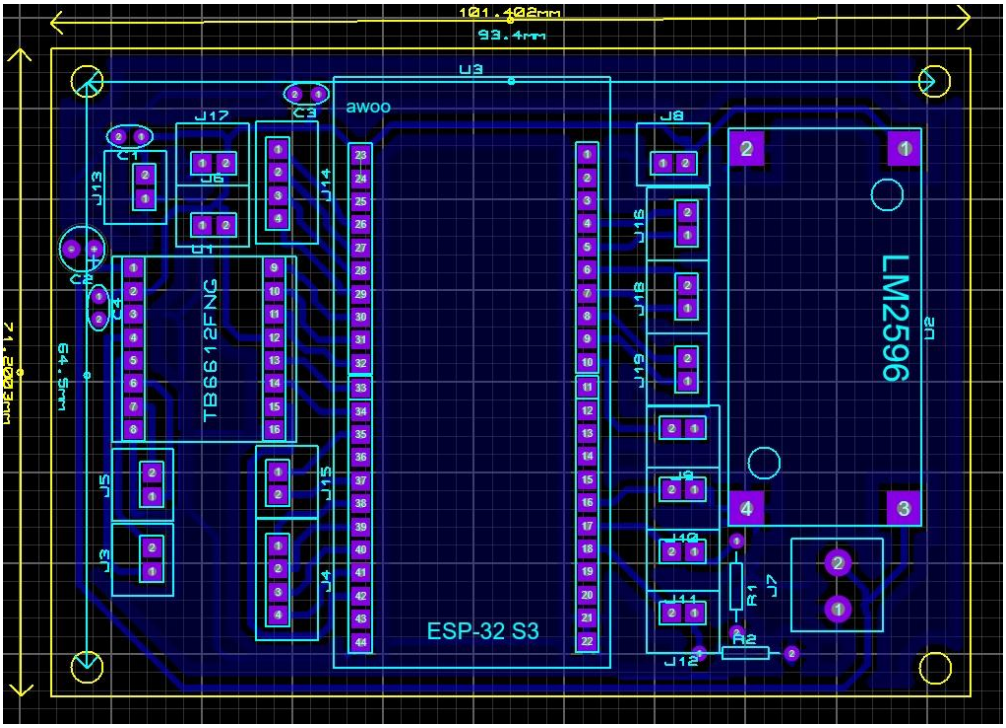


Ilustración 7. PCB ruteo

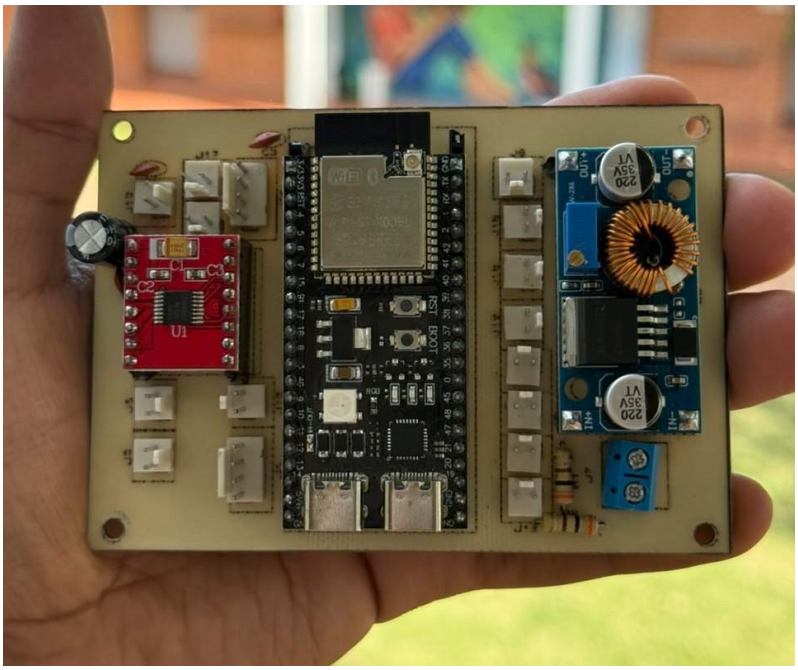


Ilustración 8. PCB montaje físico.

Asignación de pines y parámetros físicos del carro

```
Ecoder Izquierdo
fase A -- GPIO 42
fase B -- GPIO 41

Ecoder Derecho
fase A -- GPIO 40
fase B -- GPIO 39

Pulsador Calibrar -- GPIO 38
Led calibrar -- GPIO 47

Pulsador START -- GPIO 48
Led Star -- GPIO 21

Puente H TB6612
Pwma -- GPIO15
Ain1 -- GPIO 17
Ain2 -- GPIO 16
Stby -- 3.3v
Bin1 -- GPIO 18
Bin2 -- GPIO 8
Pwmb -- GPIO14

QTRA
d1 -- GPIO 4
d2 -- GPIO 5
d3 -- GPIO 6
d4 -- GPIO 7
d5 -- GPIO 3
d6 -- GPIO 9
d7 -- GPIO 2
d8 -- GPIO 1

GIROSCOPIO
SCK -- GPIO 12
MOSI -- GPIO 11
MISO -- GPIO 13
CS -- GPIO 10
```

5. IMPLEMENTACIÓN DEL CONTROL DE EQUILIBRIO

5.1. Diagrama de Flujo del Algoritmo Implementado

El programa corre sobre un ESP32 con FreeRTOS y se organiza en tres bloques: **setup()**, que configura el hardware una sola vez; la tarea **TaskBalanceControl**, que ejecuta el control de equilibrio cada 5 ms; y **loop()**, que atiende la comunicación WebSocket y envía la telemetría cada 50 ms. La Figura 1 muestra el arranque y la telemetría, y la Figura 2 el detalle de la tarea de control.

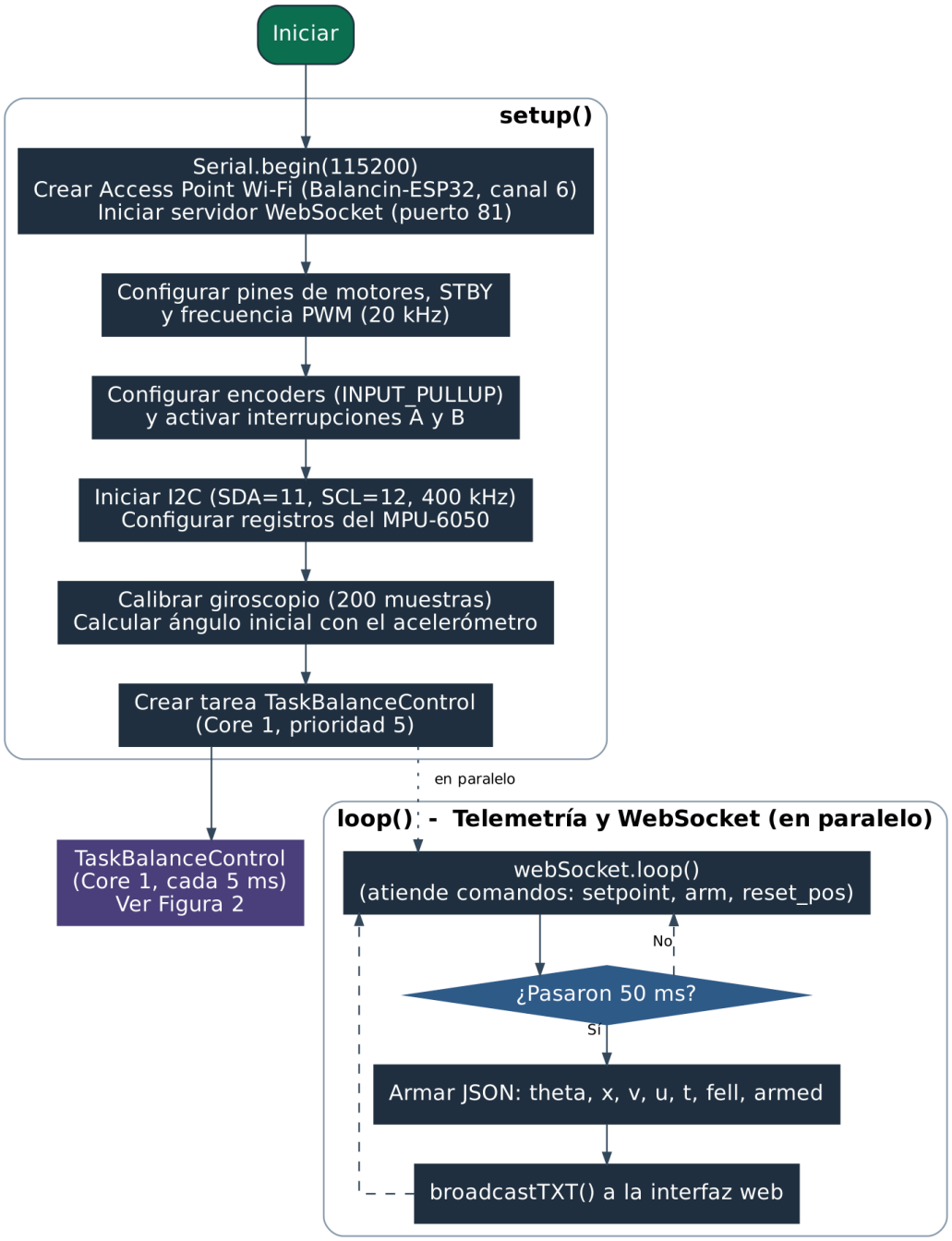


Figura 1. Diagrama de flujo: arranque (setup) y telemetría (loop).

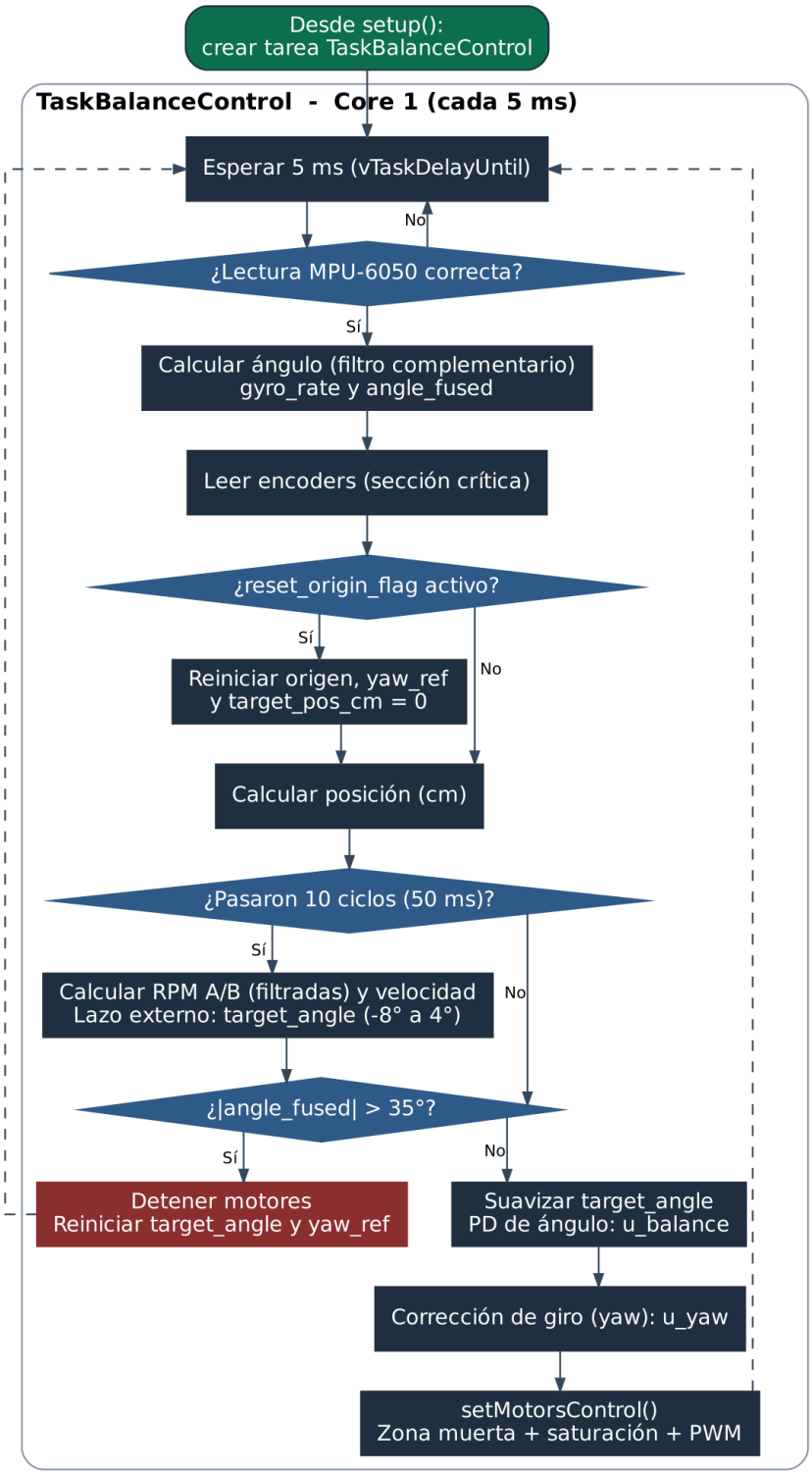


Figura 2. Diagrama de flujo de la tarea de control TaskBalanceControl (cada 5 ms).

5.2. Desglose del Código del Algoritmo

5.2.1. Librerías (#include)

```
#include <Arduino.h>
#include <Wire.h>
#include <WiFi.h>
#include <WebSocketsServer.h>
#include <ArduinoJson.h>
```

Se incluyen las librerías necesarias para el hardware, la comunicación y el formato de datos:

Librería	Función en el programa
`Arduino.h`	Núcleo del framework Arduino: `pinMode`, `digitalWrite`, `analogWrite`, `millis`, `constrain`, interrupciones y tipos de datos básicos.
`Wire.h`	Comunicación **I2C** con el sensor inercial MPU-6050 (lectura de aceleración y velocidad angular).
`WiFi.h`	Permite que el ESP32 cree su propia red Wi-Fi (modo Access Point) para conectarse desde el computador o el celular.
`WebSocketsServer.h`	Servidor **WebSocket** (puerto 81): canal bidireccional en tiempo real para recibir comandos y enviar telemetría a la interfaz web.
`ArduinoJson.h`	Serializa y deserializa mensajes en formato **JSON**, que es el formato de los comandos recibidos y de la telemetría enviada.

Además, el código usa directamente funciones de FreeRTOS incluidas en el núcleo del ESP32 (`xTaskCreatePinnedToCore`, `vTaskDelayUntil`, `portENTER_CRITICAL`) para ejecutar el control con temporización exacta.

5.2.2. Configuración Wi-Fi y WebSockets

```
const char* ssid = "Balancin-ESP32";
WebSocketsServer webSocket = WebSocketsServer(81);
```

`ssid` es el nombre de la red Wi-Fi que crea el robot. `webSocket` es el objeto servidor que escucha en el **puerto 81**. Con ellos el robot funciona de forma autónoma, sin necesidad de un router externo: el usuario se conecta a la red “Balancin-ESP32” y abre la interfaz web de monitoreo y control.

5.2.3. Asignación de pines y parámetros físicos del carro

```
#define PIN_I2C_SDA          11
#define PIN_I2C_SCL          12
#define I2C_CLOCK_SPEED      400000

#define PWMA                 15
#define AIN1                 17
#define AIN2                 16
const int ENC_A_A          = 41;
const int ENC_A_B          = 42;

#define PWMB                 14
#define BIN1                 18
#define BIN2                 8
const int ENC_B_A          = 40;
const int ENC_B_B          = 39;

#define STBY                 21
#define PWM_FREQ_HZ          20000
```

En esta sección se asignan los pines del ESP32 a cada periférico. Los pines **AIN1/AIN2** y **BIN1/BIN2** definen el sentido de giro de cada motor a través del puente H; **PWMA** y **PWMB** controlan la velocidad mediante modulación por ancho de pulso; y **STBY** habilita o deshabilita el driver de motores. Cada motor tiene un encoder de dos canales (A y B) para medir giro y dirección.

Función	Pines (GPIO)	Descripción
Bus I2C (MPU-6050)	SDA = 11, SCL = 12	Reloj a 400 kHz (modo rápido).
Motor A	PWMA = 15, AIN1 = 17, AIN2 = 16	Velocidad por PWM y dirección por AIN1/AIN2.
Encoder A	ENC_A_A = 41, ENC_A_B = 42	Canal A genera la interrupción; canal B indica el sentido.
Motor B	PWMB = 14, BIN1 = 18, BIN2 = 8	Velocidad por PWM y dirección por BIN1/BIN2.
Encoder B	ENC_B_A = 40, ENC_B_B = 39	Canal A genera la interrupción; canal B indica el sentido.
Habilitación del driver	STBY = 21	En alto (HIGH) activa el puente H.
Frecuencia PWM	20 kHz	Por encima del rango audible: los motores no “silban”.

Parámetros mecánicos y de conversión de los encoders

```
const float PULSES_PER_REV     = 450.0f;
const float WHEEL_DIAMETER_CM  = 6.6f;
const float WHEEL_PERIMETER_CM = PI * WHEEL_DIAMETER_CM;
const float CM_PER_PULSE       = WHEEL_PERIMETER_CM / PULSES_PER_REV;

const int DIR_ENC_A            = 1;
const int DIR_ENC_B            = -1;

volatile long encoderTicksA    = 0;
volatile long encoderTicksB    = 0;
```

- **PULSES_PER_REV = 450:** pulsos del encoder por cada vuelta completa de la rueda.
- **WHEEL_DIAMETER_CM = 6.6:** diámetro de la rueda; con él se calcula el perímetro ($\pi \cdot d \approx$ 20,73 cm).
- **CM_PER_PULSE:** distancia que avanza el robot por cada pulso (≈ 0,046 cm). Convierte los pulsos acumulados en centímetros recorridos.
- **DIR_ENC_A / DIR_ENC_B:** signo de cada encoder. El motor B está montado de forma espejada, por lo que su conteo se invierte (−1) para que ambas ruedas cuenten positivo al avanzar.
- **encoderTicksA / encoderTicksB:** contadores de pulsos. Son `volatile` porque los modifica una interrupción y los lee la tarea de control.

5.2.4. Registros y estructura de datos del MPU-6050

```
#define RAD2GRAD             57.2957795f
#define MPU6050_I2C_ADDR     0x68
#define REG_SMPLRT_DIV       0x19
#define REG_CONFIG           0x1A
#define REG_GYRO_CONFIG      0x1B
#define REG_ACCEL_CONFIG     0x1C
#define REG_ACCEL_XOUT_H     0x3B
#define REG_PWR_MGMT_1       0x6B

typedef union {
```

```
  struct { uint8_t x_accel_h, x_accel_l, y_accel_h, y_accel_l, z_accel_h, z_accel_l,
                   t_h, t_l, x_gyro_h, x_gyro_l, y_gyro_h, y_gyro_l, z_gyro_h, z_gyro_l;
} reg;
  struct { int16_t x_accel, y_accel, z_accel, temp, x_gyro, y_gyro, z_gyro; } value;
} MPU_Data;

MPU_Data mpu;
float x_gyro_offset = 0.0f;
float angle_fused   = 0.0f;
float gyro_rate     = 0.0f;
```

El MPU-6050 se maneja directamente por registros I2C, sin librería externa. Se definen la dirección del sensor (**0x68**) y los registros que se usan:

Registro	Dirección	Uso
`REG_PWR_MGMT_1`	0x6B	Administración de energía: reinicio (0x80) y selección del reloj del giroscopio (0x01).
`REG_CONFIG`	0x1A	Filtro pasa-bajos digital (DLPF). Se escribe 0x03 para reducir el ruido de las lecturas.
`REG_GYRO_CONFIG`	0x1B	Rango del giroscopio. Se escribe 0x08 → ±500 °/s (sensibilidad 65,5 LSB por °/s).
`REG_ACCEL_CONFIG`	0x1C	Rango del acelerómetro. Se escribe 0x00 → ±2 g (16384 LSB por g).
`REG_SMPLRT_DIV`	0x19	Divisor de la frecuencia de muestreo. Con valor 4 el sensor entrega ≈ 200 Hz (una muestra cada 5 ms).
`REG_ACCEL_XOUT_H`	0x3B	Primer registro de datos; desde aquí se leen de corrido 14 bytes (acelerómetro, temperatura y giroscopio).

- **RAD2GRAD = 57,2957795:** factor para convertir radianes a grados (180/π).
- **MPU_Data (union):** permite ver los mismos 14 bytes leídos del sensor de dos formas: como bytes individuales (`reg`, parte alta y baja de cada dato) o como enteros de 16 bits ya ensamblados (`value`).
- **x_gyro_offset:** sesgo (bias) del giroscopio, medido durante la calibración inicial.
- **angle_fused:** ángulo de inclinación estimado (resultado del filtro complementario), en grados.
- **gyro_rate:** velocidad angular del robot en °/s, ya con el offset restado.

5.2.5. Parámetros de control, offset físico y variables globales

```
volatile float Kp_angle      = 24.0f;
volatile float Kd_angle      = 0.15f;
volatile float Kp_speed      = -0.05f;
volatile float Ki_speed      = 0.0f;
volatile float Kp_yaw        = 0.0f;

const float    PHYSICAL_BAL_OFFSET = -2.0f;
volatile float user_target_angle   = 0.0f;
volatile float target_pos_cm       = 0.0f;

volatile int   deadband_pwm        = 10;
const float    MAX_FALL_ANGLE      = 35.0f;
const float    MAX_POS_ERROR_CM    = 40.0f;

volatile float target_angle        = 0.0f;
volatile float target_angle_smooth = 0.0f;
volatile float current_pos_cm      = 0.0f;
volatile float current_rpmA        = 0.0f;
```

```
volatile float current_rpmB        = 0.0f;
volatile float current_speed_rpm   = 0.0f;
volatile int   current_pwm         = 0;
volatile bool  reset_origin_flag   = false;
volatile bool  motors_armed        = true;

portMUX_TYPE controlMux = portMUX_INITIALIZER_UNLOCKED;
```

El control es una **cascada de dos lazos**: un lazo interno rápido que mantiene el ángulo (PD) y un lazo externo lento que ajusta el ángulo deseado según la velocidad y la posición. Estas son las ganancias y parámetros que lo definen:

Parámetro	Valor	Descripción
`Kp_angle`	24,0	Ganancia proporcional del lazo de ángulo: cuánto PWM se aplica por cada grado de error de inclinación.
`Kd_angle`	0,15	Ganancia derivativa del lazo de ángulo: se multiplica por la velocidad angular del giroscopio y amortigua las oscilaciones.
`Kp_speed`	−0,05	Ganancia del lazo externo sobre la velocidad (RPM): modifica el ángulo objetivo para frenar el avance o retroceso del robot.
`Ki_speed`	0,0	Ganancia del lazo externo sobre el error de posición (cm). Con valor 0 está desactivado; a pesar de su nombre, multiplica la posición, no una integral.
`Kp_yaw`	0,0	Ganancia de corrección de giro (yaw) para que el robot no se desvíe. Con valor 0 está desactivada.
`PHYSICAL_BAL_OFFSET`	−2,0°	Compensación del peso: el punto de equilibrio real del carro no está en 0°, sino en −2° por la distribución de la masa.
`user_target_angle`	0,0°	Ángulo adicional pedido desde la web (0 = quedarse en equilibrio; otros valores producen avance o retroceso).
`target_pos_cm`	0,0 cm	Posición objetivo pedida desde la web.
`deadband_pwm`	10	Zona muerta: PWM mínimo que se suma para vencer la fricción estática de los motores.
`MAX_FALL_ANGLE`	35°	Ángulo máximo permitido; si se supera, el robot se considera caído y se detienen los motores.
`MAX_POS_ERROR_CM`	40 cm	Límite (saturación) del error de posición que usa el lazo externo.

Las demás variables almacenan el estado en tiempo real y se comparten entre la tarea de control y el loop de telemetría: `target_angle` (ángulo deseado calculado), `target_angle_smooth` (versión suavizada), `current_pos_cm` (posición recorrida), `current_rpmA/B` y `current_speed_rpm` (velocidad de las ruedas y promedio), `current_pwm` (salida aplicada), `reset_origin_flag` (orden de reiniciar la distancia) y `motors_armed` (habilitación de los motores).

Todas se declaran **volatile** porque son modificadas desde varios contextos (interrupciones, tarea de control y callback del WebSocket). **controlMux** es un mutex tipo *spinlock* que protege la lectura de los contadores de los encoders mientras una interrupción podría modificarlos.

5.2.6. Funciones de bajo nivel: encoders, I2C y lectura del sensor

a) Interrupciones de los encoders

```
void IRAM_ATTR handleEncoderA() {
  if (digitalRead(ENC_A_A) == digitalRead(ENC_A_B)) encoderTicksA += DIR_ENC_A;
  else encoderTicksA -= DIR_ENC_A;
}
```

```
void IRAM_ATTR handleEncoderB() {
  if (digitalRead(ENC_B_A) == digitalRead(ENC_B_B)) encoderTicksB += DIR_ENC_B;
  else encoderTicksB -= DIR_ENC_B;
}
```

Cada vez que el canal A de un encoder cambia de nivel (flanco de subida o bajada), se ejecuta la interrupción. Allí se compara el canal A con el canal B: si son iguales, el motor gira en un sentido y se suma un pulso; si son distintos, gira en el sentido contrario y se resta. Así se obtiene un conteo con signo que indica avance y dirección (decodificación en cuadratura). `IRAM_ATTR` coloca la función en la memoria RAM interna para que la interrupción sea rápida y segura.

b) Comunicación I2C

```
int i2c_write(uint8_t reg, uint8_t val) {
  Wire.beginTransmission(MPU6050_I2C_ADDR);
  Wire.write(reg); Wire.write(val);
  return Wire.endTransmission();
}
int i2c_read(uint8_t reg, uint8_t *buf, uint8_t len) {
  Wire.beginTransmission(MPU6050_I2C_ADDR); Wire.write(reg);
  if (Wire.endTransmission(false) != 0) return -1;
  if (Wire.requestFrom((uint8_t)MPU6050_I2C_ADDR, len, (uint8_t)true) != len) return -2;
  for (uint8_t i = 0; i < len; i++) buf[i] = Wire.read();
  return 0;
}
```

- **i2c_write:** escribe un valor en un registro del MPU-6050 (se usa para configurarlo). Devuelve 0 si la transmisión fue exitosa.
- **i2c_read:** lee `len` bytes consecutivos a partir de un registro. Devuelve 0 si todo salió bien, −1 si falló el envío de la dirección y −2 si el sensor no entregó la cantidad de bytes pedida.

c) Lectura del sensor

```
bool mpu_read() {
  if (i2c_read(REG_ACCEL_XOUT_H, (uint8_t *)&mpu, sizeof(mpu)) != 0) return false;
  #define SWAP_B(a, b) { uint8_t t = a; a = b; b = t; }
  SWAP_B(mpu.reg.x_accel_h, mpu.reg.x_accel_l); SWAP_B(mpu.reg.y_accel_h,
mpu.reg.y_accel_l);
  SWAP_B(mpu.reg.z_accel_h, mpu.reg.z_accel_l); SWAP_B(mpu.reg.x_gyro_h,
mpu.reg.x_gyro_l);
  return true;
}
```

Lee de una sola vez los 14 bytes de datos del sensor y los guarda en la estructura `mpu`. El MPU-6050 envía cada dato de 16 bits con el byte alto primero (*big-endian*), mientras que el ESP32 los interpreta con el byte bajo primero (*little-endian*); por eso la macro `SWAP_B` intercambia los dos bytes de cada dato que se utiliza (aceleración en X, Y, Z y giro en X). La función devuelve `false` si la lectura falla, lo que permite a la tarea de control ignorar ese ciclo.

5.2.7. Control de motores

```
int applyDeadbandAndClamp(float u) {
  int pwm_out = 0;
  if (u > 0.05f) pwm_out = (int)roundf(u) + deadband_pwm;
  else if (u < -0.05f) pwm_out = (int)roundf(u) - deadband_pwm;
  return constrain(pwm_out, -255, 255);
}

void driveSingleMotor(uint8_t pinPWM, uint8_t pinIN1, uint8_t pinIN2, int pwm) {
  if (!motors_armed) pwm = 0;
```

```
  if (pwm > 0) { digitalWrite(pinIN1, HIGH); digitalWrite(pinIN2, LOW);
analogWrite(pinPWM, pwm); }
  else if (pwm < 0) { digitalWrite(pinIN1, LOW); digitalWrite(pinIN2, HIGH);
analogWrite(pinPWM, -pwm); }
  else { digitalWrite(pinIN1, LOW); digitalWrite(pinIN2, LOW); analogWrite(pinPWM, 0); }
}

void stopMotors() {
  current_pwm = 0; analogWrite(PWMA, 0); analogWrite(PWMB, 0);
  digitalWrite(AIN1, LOW); digitalWrite(AIN2, LOW); digitalWrite(BIN1, LOW);
digitalWrite(BIN2, LOW);
}

void setMotorsControl(float u_balance, float u_yaw) {
  int pwmA = applyDeadbandAndClamp(u_balance - u_yaw);
  int pwmB = applyDeadbandAndClamp(u_balance + u_yaw);
  if (!motors_armed) { pwmA = 0; pwmB = 0; }
  current_pwm = (pwmA + pwmB) / 2;
  driveSingleMotor(PWMA, AIN1, AIN2, pwmA);
  driveSingleMotor(PWMB, BIN1, BIN2, pwmB);
}
```

- **applyDeadbandAndClamp:** convierte la salida del controlador en un valor de PWM utilizable. Si la señal es mayor que ±0,05 le suma la zona muerta (10) en su mismo sentido, para que el motor realmente se mueva; si es casi cero, la salida es 0. Finalmente limita el resultado al rango −255 a 255.
- **driveSingleMotor:** mueve un motor. Si el PWM es positivo activa IN1 y desactiva IN2 (un sentido); si es negativo invierte los pines (sentido contrario) y usa el valor absoluto como velocidad; si es cero apaga ambas entradas. Si los motores están desarmados, fuerza el PWM a 0.
- **stopMotors:** apaga por completo ambos motores (PWM en 0 y todas las entradas del puente H en bajo).
- **setMotorsControl:** recibe la señal de equilibrio y la de giro, y calcula el PWM de cada rueda: `pwmA = u_balance - u_yaw` y `pwmB = u_balance + u_yaw`. La parte común (balance) mueve ambas ruedas igual, y la diferencia (yaw) las hace girar distinto para corregir la dirección. También guarda el PWM promedio en `current_pwm` para la telemetría.

5.2.8. Recepción de comandos desde la web

```
void webSocketEvent(uint8_t num, WStype_t type, uint8_t * payload, size_t length) {
  if (type == WStype_TEXT) {
    StaticJsonDocument<200> doc;
    DeserializationError error = deserializeJson(doc, payload);
    if (!error) {
      String cmd = doc["cmd"];
      if (cmd == "setpoint") {
        if (doc.containsKey("angle")) user_target_angle = doc["angle"];
        if (doc.containsKey("pos")) target_pos_cm = doc["pos"];
      }
      else if (cmd == "arm") {
        motors_armed = doc["on"];
        if(!motors_armed) stopMotors();
      }
      else if (cmd == "reset_pos") {
        reset_origin_flag = true;
      }
    }
  }
}
```

Esta función es el *callback* del WebSocket: se ejecuta cada vez que la interfaz web envía un mensaje. Si el mensaje es texto, se interpreta como JSON y se lee el campo `cmd` para decidir la acción:

Comando	Ejemplo de mensaje	Efecto
`setpoint`	{"cmd":"setpoint","angle":1.5,"pos":10}	Actualiza el ángulo adicional pedido por el usuario y/o la posición objetivo (campos opcionales).
`arm`	{"cmd":"arm","on":true}	Arma (true) o desarma (false) los motores. Al desarmar se detienen de inmediato.
`reset_pos`	{"cmd":"reset_pos"}	Solicita reiniciar el origen de posición; la tarea de control lo ejecuta en su siguiente ciclo.

Si el JSON es inválido, el mensaje simplemente se ignora (`if (!error)`).

5.2.9. Tarea de control principal (TaskBalanceControl)

Es el corazón del algoritmo. Se ejecuta como una tarea de FreeRTOS fijada al **Core 1** con prioridad 5, de modo que el equilibrio no se ve afectado por el Wi-Fi ni por la comunicación. Cada ciclo dura exactamente 5 ms (200 Hz). A continuación se explica por etapas.

a) Temporización y variables locales

```
void TaskBalanceControl(void *pvParameters) {
  const TickType_t xPeriod = pdMS_TO_TICKS(5);
  TickType_t xLastWakeTime = xTaskGetTickCount();
  const float dt = 0.005f;

  uint8_t loop_counter = 0;
  long last_enc_A = 0, last_enc_B = 0;
  long origin_ticks_sum = 0, yaw_ref_diff = 0;

  for (;;) {
    vTaskDelayUntil(&xLastWakeTime, xPeriod);
    if (mpu_read()) {
      ...
    }
  }
}
```

`vTaskDelayUntil` garantiza un periodo fijo de 5 ms entre ciclos (a diferencia de `delay`, que acumularía el tiempo de cálculo). Esto permite usar un `dt` constante de 0,005 s en el filtro. Las variables locales guardan: `loop_counter` (cuenta ciclos para el lazo lento de 50 ms), `last_enc_A/B` (conteos anteriores para calcular velocidad), `origin_ticks_sum` (origen de la posición) y `yaw_ref_diff` (diferencia de referencia entre ruedas para mantener la dirección).

b) Medición del ángulo: filtro complementario

```
float accel_angle = atan2f((float)mpu.value.y_accel, (float)mpu.value.z_accel) *
RAD2GRAD;
gyro_rate = ((float)mpu.value.x_gyro - x_gyro_offset) / 65.5f;
angle_fused = 0.995f * (angle_fused + gyro_rate * dt) + 0.005f * accel_angle;
```

- **accel_angle:** ángulo calculado con el acelerómetro, usando la arcotangente de las componentes Y y Z de la gravedad. Es estable en el largo plazo pero ruidoso ante vibraciones.
- **gyro_rate:** velocidad angular en °/s: se resta el offset de calibración y se divide por 65,5 (sensibilidad del giroscopio en ±500 °/s).
- **angle_fused:** filtro complementario. Integra la velocidad del giroscopio (rápido y suave, pero con deriva) y lo corrige lentamente con el acelerómetro (sin deriva, pero ruidoso). La ponderación es **99,5 % giroscopio + 0,5 % acelerómetro**.

c) Lectura de encoders y posición

```
long cur_A, cur_B;
portENTER_CRITICAL(&controlMux);
cur_A = encoderTicksA; cur_B = encoderTicksB;
portEXIT_CRITICAL(&controlMux);

if (reset_origin_flag) {
  origin_ticks_sum = cur_A + cur_B;
  yaw_ref_diff     = cur_A - cur_B;
  target_pos_cm    = 0.0f;
  reset_origin_flag = false;
}

float avg_ticks_from_origin = 0.5f * (float)((cur_A + cur_B) - origin_ticks_sum);
current_pos_cm = avg_ticks_from_origin * CM_PER_PULSE;
```

Se copian los contadores de los encoders dentro de una **sección crítica**, para que una interrupción no los cambie a mitad de la lectura. Si la web pidió reiniciar la distancia (`reset_origin_flag`), se toma la posición actual como nuevo origen, se actualiza la referencia de giro y se pone la posición objetivo en 0. Luego la posición del robot se calcula como el promedio de los pulsos de ambas ruedas desde el origen, multiplicado por los centímetros por pulso.

d) Velocidad de las ruedas (cada 50 ms)

```
loop_counter++;
if (loop_counter >= 10) {
  loop_counter = 0;
  long delta_A = cur_A - last_enc_A; long delta_B = cur_B - last_enc_B;
  last_enc_A = cur_A; last_enc_B = cur_B;

  float raw_rpmA = ((float)delta_A / PULSES_PER_REV) * (60.0f / 0.05f);
  float raw_rpmB = ((float)delta_B / PULSES_PER_REV) * (60.0f / 0.05f);
  current_rpmA = 0.70f * current_rpmA + 0.30f * raw_rpmA;
  current_rpmB = 0.70f * current_rpmB + 0.30f * raw_rpmB;
  current_speed_rpm = 0.5f * (current_rpmA + current_rpmB);
  ...
```

Esta parte se ejecuta una de cada 10 veces (10 × 5 ms = 50 ms), con lo que se forma el **lazo lento**. Se calcula cuántos pulsos avanzó cada rueda en esos 50 ms, se convierte a revoluciones por minuto (RPM) y se suaviza con un filtro exponencial (70 % valor anterior + 30 % valor nuevo) para eliminar el ruido de cuantización. La velocidad del robot es el promedio de ambas ruedas.

e) Lazo externo: cálculo del ángulo objetivo

```
if (fabs(angle_fused) <= MAX_FALL_ANGLE) {
  float error_pos_cm = current_pos_cm - target_pos_cm;
  float pos_cm_clamped = constrain(error_pos_cm, -MAX_POS_ERROR_CM, MAX_POS_ERROR_CM);

  float offset_compensado = PHYSICAL_BAL_OFFSET + user_target_angle;
  float cmd_angle = (current_speed_rpm * Kp_speed) + (pos_cm_clamped * Ki_speed) +
offset_compensado;

  target_angle = constrain(cmd_angle, -8.0f, 4.0f);
}
```

Mientras el robot no esté caído, se calcula el ángulo al que debe inclinarse. Es la suma de tres términos: la corrección por velocidad (`current_speed_rpm · Kp_speed`), la corrección por posición (`error · Ki_speed`, saturada a ±40 cm) y el **offset físico** de −2° más el ángulo pedido por el usuario. Al final se limita a un rango de −8° a +4° por seguridad. Con esto el robot, además de equilibrarse, puede frenar su desplazamiento y responder a los comandos de la web.

f) Condición de seguridad (robot caído)

```
if (fabs(angle_fused) > MAX_FALL_ANGLE) {
  stopMotors();
  target_angle        = PHYSICAL_BAL_OFFSET + user_target_angle;
  target_angle_smooth = target_angle;
  yaw_ref_diff        = cur_A - cur_B;
  continue;
}
```

Si el robot se inclina más de **35°**, se detienen los motores automáticamente para evitar daños o movimientos bruscos. Además se reinician el ángulo objetivo y la referencia de giro, y `continue` salta el resto del ciclo (no se calcula el PID). Cuando el robot vuelve a quedar dentro del rango, el control se reanuda de forma automática.

g) Lazo interno: control PD del ángulo

```
target_angle_smooth = 0.90f * target_angle_smooth + 0.10f * target_angle;
float error_angle = angle_fused - target_angle_smooth;
float u_balance = (error_angle * Kp_angle) + (gyro_rate * Kd_angle);
```

- **target_angle_smooth:** filtro que suaviza los cambios del ángulo objetivo (90 % anterior + 10 % nuevo), evitando tirones cuando el lazo externo lo actualiza.
- **error_angle:** diferencia entre el ángulo medido y el ángulo deseado.
- **u_balance:** salida del controlador. Término proporcional (`error · Kp_angle`) más término derivativo (`gyro_rate · Kd_angle`). La derivada se toma directamente de la velocidad angular del giroscopio, que es más limpia que derivar numéricamente el ángulo. En este lazo no se usa acción integral.

h) Corrección de giro (yaw)

```
long yaw_error = (cur_A - cur_B) - yaw_ref_diff;
float u_yaw = (float)yaw_error * Kp_yaw;
u_yaw = constrain(u_yaw, -30.0f, 30.0f);
```

Mide cuánto se ha desviado una rueda respecto a la otra (diferencia de pulsos menos la referencia) y genera una corrección proporcional, limitada a ±30. Con `Kp_yaw = 0` esta corrección está desactivada en la configuración actual, pero la estructura queda lista para activarla.

i) Actuación

```
setMotorsControl(u_balance, u_yaw);
```

Finalmente se envían las señales de balance y giro a los motores. La función aplica la zona muerta, la saturación y el sentido de giro de cada rueda (ver sección 5.2.7).

5.2.10. Función setup()

Se ejecuta una sola vez al encender el robot y deja todo listo antes de iniciar el control. Sus pasos son:

1. Comunicación serial y red Wi-Fi

```
Serial.begin(115200); delay(1000);
Serial.println("\nConfigurando AP Wi-Fi en el canal 6...");
WiFi.softAP(ssid, NULL, 6);

webSocket.begin();
webSocket.onEvent(webSocketEvent);
```

Se inicia el puerto serie a 115200 baudios (para mensajes de depuración), se crea la red Wi-Fi “Balancin-ESP32” en el canal 6 y **sin contraseña** (`NULL`), y se arranca el servidor WebSocket asociándole la función `webSocketEvent` como manejador de mensajes.

2. Configuración de pines, PWM y encoders

```
pinMode(PWMA, OUTPUT); pinMode(AIN1, OUTPUT); pinMode(AIN2, OUTPUT);
pinMode(PWMB, OUTPUT); pinMode(BIN1, OUTPUT); pinMode(BIN2, OUTPUT);
analogWriteFrequency(PWMA, PWM_FREQ_HZ); analogWriteFrequency(PWMB, PWM_FREQ_HZ);
#ifdef STBY
pinMode(STBY, OUTPUT); digitalWrite(STBY, HIGH);
#endif
pinMode(ENC_A_A, INPUT_PULLUP); pinMode(ENC_A_B, INPUT_PULLUP);
pinMode(ENC_B_A, INPUT_PULLUP); pinMode(ENC_B_B, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(ENC_A_A), handleEncoderA, CHANGE);
attachInterrupt(digitalPinToInterrupt(ENC_B_A), handleEncoderB, CHANGE);
```

Los pines del puente H se configuran como salidas y se fija la frecuencia del PWM en 20 kHz. El pin STBY se pone en alto para habilitar el driver. Los pines de los encoders se configuran como entrada con resistencia *pull-up* interna, y se asocian las interrupciones `handleEncoderA/B` al canal A de cada encoder, con disparo en **cualquier cambio de nivel** (`CHANGE`).

3. Inicialización del sensor MPU-6050

```
Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL, I2C_CLOCK_SPEED);
i2c_write(REG_PWR_MGMT_1, 0x80); delay(100);
i2c_write(REG_PWR_MGMT_1, 0x01); i2c_write(REG_CONFIG, 0x03);
i2c_write(REG_GYRO_CONFIG, 0x08); i2c_write(REG_ACCEL_CONFIG, 0x00);
i2c_write(REG_SMPLRT_DIV, 4);
```

Se inicia el bus I2C en los pines 11 y 12 a 400 kHz. Se reinicia el sensor (0x80) y se espera 100 ms; luego se despierta seleccionando el reloj del giroscopio (0x01), se activa el filtro digital (0x03), se configura el giroscopio a ±500 °/s (0x08), el acelerómetro a ±2 g (0x00) y se ajusta el muestreo a ≈ 200 Hz (divisor 4).

4. Calibración del giroscopio y ángulo inicial

```
Serial.println("Calibrando IMU... Manten el robot inmovil.");
int32_t sum = 0;
for (int i = 0; i < 200; i++) { mpu_read(); sum += mpu.value.x_gyro; delay(10); }
x_gyro_offset = (float)sum / 200.0f;
mpu_read(); angle_fused = atan2f((float)mpu.value.y_accel, (float)mpu.value.z_accel) *
RAD2GRAD;
```

Con el robot **inmóvil** se toman 200 lecturas del giroscopio (durante unos 2 segundos) y se calcula su promedio, que corresponde al error de offset (bias) del sensor. Ese valor se resta después en cada lectura. Por último se calcula el ángulo inicial a partir del acelerómetro para que el filtro complementario arranque desde la posición real del robot.

5. Creación de la tarea de control

```
xTaskCreatePinnedToCore(TaskBalanceControl, "BalControl", 4096, NULL, 5, NULL, 1);
```

Se crea la tarea de control con nombre “BalControl”, 4096 bytes de pila, prioridad 5, y se fija al **Core 1** del procesador. A partir de aquí el robot comienza a equilibrarse.

5.2.11. Función loop(): WebSocket y telemetría

```
void loop() {
  webSocket.loop();
```

```
  static uint32_t last_telemetry_time = 0;
  if (millis() - last_telemetry_time >= 50) { // 20 Hz
    last_telemetry_time = millis();

    StaticJsonDocument<256> doc;
    doc["theta"] = angle_fused;
    doc["x"]     = current_pos_cm;
    doc["v"]     = current_speed_rpm * (WHEEL_PERIMETER_CM / 60.0f);
    doc["u"]     = (float)current_pwm * 100.0f / 255.0f;
    doc["t"]     = millis();
    doc["fell"]  = fabs(angle_fused) > MAX_FALL_ANGLE;
    doc["armed"] = motors_armed && (fabs(angle_fused) <= MAX_FALL_ANGLE);

    char jsonBuffer[256];
    serializeJson(doc, jsonBuffer);
    webSocket.broadcastTXT(jsonBuffer);
  }
}
```

`loop()` corre en paralelo a la tarea de control y no participa en el equilibrio. Hace dos cosas: atender el servidor WebSocket (`webSocket.loop()`, que recibe los comandos) y, cada 50 ms (**20 Hz**), armar un mensaje JSON con el estado del robot y enviarlo a todos los clientes conectados (`broadcastTXT`). El uso de `millis()` en lugar de `delay()` evita bloquear la comunicación.

Campo	Unidad	Significado
`theta`	grados	Ángulo de inclinación del robot (angle_fused).
`x`	cm	Posición recorrida desde el origen.
`v`	cm/s	Velocidad lineal: RPM promedio × perímetro de la rueda / 60.
`u`	% (±100)	Esfuerzo de control: PWM promedio expresado como porcentaje del máximo (255).
`t`	ms	Tiempo desde el encendido (`millis()`).
`fell`	booleano	Verdadero si |ángulo| supera 35° (robot caído).
`armed`	booleano	Verdadero si los motores están armados y el robot no está caído.

Nota sobre los núcleos: en los comentarios del código `loop()` aparece como “Core 0”. En Arduino-ESP32, por defecto, `loop()` se ejecuta en el Core 1 con prioridad 1; por esa razón la tarea de control (prioridad 5, también en el Core 1) lo interrumpe siempre que le corresponde ejecutarse. El diseño funciona igual, pero conviene tenerlo presente si se explica la distribución por núcleos.

5.3. Resumen de las Etapas del Lazo de Control

Etapa	Descripción
Medición (variable controlada)	El MPU-6050 entrega aceleración y velocidad angular (200 Hz). El filtro complementario (99,5 % giroscopio, 0,5 % acelerómetro) estima el ángulo de inclinación `angle_fused`. Los encoders miden posición y velocidad.
Control — lazo externo (50 ms)	A partir de la velocidad y la posición calcula el ángulo objetivo: `target_angle` = Kp_speed · RPM + Ki_speed · error de posición + offset físico (−2°) + ángulo del usuario, limitado a −8° / +4°.
Control — lazo interno (5 ms)	Controlador PD de ángulo: `u_balance` = Kp_angle · (ángulo − objetivo suavizado) + Kd_angle · velocidad angular, con Kp = 24 y Kd = 0,15.
Actuación (variable manipulada)	PWM de cada motor = u_balance ∓ u_yaw, con zona muerta de 10, saturación ±255 y sentido de giro por el puente H.

Etapa	Descripción
Seguridad	Si |ángulo| > 35° se detienen los motores; también se detienen al desarmar desde la web (`arm` = false).
Comunicación	Comandos JSON recibidos por WebSocket (setpoint, arm, reset_pos) y telemetría a 20 Hz (theta, x, v, u, t, fell, armed).

REFERENCIAS.

Autodesk. (2026). *Autodesk Fusion 360* [Software de diseño asistido por computadora]. Autodesk Inc. https://www.autodesk.com/products/fusion-360

Briceño Rojas, I. K. (2026). *Documentación técnica, esquemáticos y archivos CAD del Carrito Balancín Motilón* [Carpeta compartida de Google Drive]. https://drive.google.com/drive/folders/1_sx-YP8qqcrqTymtWVOG53p-BeLnHk3x

Cabello, M., Doob, R., y colaboradores de Three.js. (2021). *Three.js: JavaScript 3D Library* (Versión r128) [Software de código abierto]. https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js

Didácticas Electrónicas S.A.S. (s. f.). *Didácticas Electrónicas: Tienda de componentes electrónicos y módulos de robótica*. Recuperado el 15 de octubre de 2026, de https://www.didacticaselectronicas.com/

Espressif Systems. (2023). *ESP32-S3 Series Datasheet: 2.4 GHz Wi-Fi and Bluetooth 5 (LE) SoC* (Versión 1.4). Espressif Systems Co., Ltd. https://www.espressif.com/en/products/socs/esp32-s3

InvenSense. (2013). *MPU-6000 and MPU-6050 Product Specification* (Revisión 3.4). InvenSense Inc.